

TEMA 5. Aseguramiento del ciclo de vida del software seguro

1 Introducción

En la era digital actual, la seguridad del software se ha convertido en una preocupación central para cualquier organización que dependa de sistemas informáticos. Los avances tecnológicos han brindado innumerables beneficios, pero también han dado lugar a amenazas cada vez más sofisticadas que pueden comprometer la integridad y confidencialidad de los datos, así como la continuidad de las operaciones. Un software inseguro no solo puede exponer a una organización a riesgos financieros y legales, sino que también puede socavar la confianza de los usuarios y clientes.

Para evitar problemas de seguridad recurrentes en una aplicación, es esencial el construir la seguridad dentro del Ciclo de Vida de Desarrollo del Software (SDLC), desarrollando estándares, políticas y guías de uso que encajen y funcionen dentro de la metodología de desarrollo. La clave para lograr un software seguro radica en el proceso de desarrollo utilizado. Es en este proceso donde se forja un producto capaz de resistir ataques anticipados, recuperarse ágilmente y mitigar el daño causado por amenazas inesperadas.

Un ciclo de desarrollo seguro no se limita a la simple adición de medidas de seguridad como una capa adicional. A nivel técnico, implica la integración de pruebas de seguridad en los procesos existentes, incorporando actividades como análisis de arquitectura, revisiones frecuentes de código y construcción de pruebas de penetración antes del lanzamiento. Sin embargo, su alcance va más allá, implicando una transformación profunda en la manera en que concebimos, diseñamos, construimos y mantenemos el software. Requiere una mentalidad proactiva que coloca la seguridad en el núcleo mismo del proceso, desde la fase de concepción hasta el despliegue y más allá.

En este contexto, este tema abordará diversos procedimientos que han modificado la manera en que se diseñan, desarrollan y programan las aplicaciones, en respuesta al creciente protagonismo de la seguridad en el ámbito del desarrollo de software. Estos esfuerzos culminan en la metodología conocida como DevSecOps, que busca integrar la seguridad de manera continua en todo el ciclo de vida del desarrollo de software.

2 Software development lifecycle (SDLC)

El SDLC (Software Development Life Cycle) o Ciclo de Vida del Desarrollo de Software es un proceso estructurado utilizado para desarrollar y mantener software de manera sistemática y efectiva. Es una metodología que guía a los equipos de desarrollo a través de las diferentes etapas del desarrollo de software, desde la concepción de la idea hasta la entrega del producto final y su mantenimiento continuo.

El SDLC se compone de varias fases interconectadas, cada una con objetivos y actividades específicas. Aunque las etapas pueden variar según la metodología utilizada, las fases típicas del SDLC son las siguientes:

- **Requisitos:** En esta fase inicial, se definen y recopilan los requisitos del software. Se identifican las necesidades del cliente y las funcionalidades que debe tener la aplicación.
- **Diseño:** En esta etapa, se elabora una especificación detallada del software. Se crea un diseño técnico y arquitectónico que servirá como base para el desarrollo.
- **Implementación:** Aquí se lleva a cabo la codificación del software según las especificaciones y el diseño previo. Los desarrolladores escriben el código y crean el producto funcional.
- **Pruebas:** En esta fase, el software se somete a pruebas exhaustivas para identificar y corregir errores, asegurándose de que el software funcione correctamente y cumpla con los requisitos establecidos.
- **Despliegue:** Una vez que el software ha sido probado y se ha asegurado su calidad, se implementa en el entorno de producción y se pone a disposición de los usuarios finales.



Figure 1: SDLC

- **Mantenimiento:** Esta etapa implica la corrección de errores, la realización de actualizaciones y mejoras, y la gestión del software a lo largo de su vida útil para garantizar su eficiencia y seguridad continua.

En esta sección, repasaremos las dos categorías principales de modelos SDLC: los clásicos y los ágiles. Cada uno aborda el proceso de desarrollo desde una perspectiva diferente, ofreciendo enfoques distintos para enfrentar los desafíos y lograr los objetivos del desarrollo de software.

2.1 Modelos clásicos

Los modelos clásicos han sido ampliamente utilizados durante décadas en el desarrollo de software. Estas metodologías suelen seguir un enfoque lineal y secuencial, con un énfasis en la planificación detallada y la ejecución paso a paso. Algunas de las más conocidas son:

- **Modelo en Cascada (Waterfall):** Es un enfoque lineal y secuencial que divide el desarrollo en etapas bien definidas, como requisitos, diseño, implementación, pruebas y mantenimiento. Cada fase debe completarse antes de pasar a la siguiente, lo que hace que sea difícil volver atrás en caso de cambios, requiriendo gran cantidad de recursos y tiempo.
- **Modelo en V (V-Model):** Es una extensión del modelo en cascada que asocia cada etapa de desarrollo con una etapa de pruebas correspondiente. Esto enfatiza la importancia de la verificación y validación en cada fase del proceso.
- **Modelo en Espiral (Spiral):** Es un enfoque iterativo que combina elementos del modelo en cascada con prototipos y evaluaciones continuas de riesgos. El desarrollo se divide en ciclos, cada uno de los cuales incluye planificación, análisis de riesgos, desarrollo, pruebas y revisión.

Los modelos clásicos tienden a ser lineales y secuenciales, lo que significa que una vez que una etapa ha sido completada, es difícil o costoso volver atrás y realizar cambios. Esto puede ser problemático cuando surgen nuevos requisitos o se descubren errores más adelante en el proceso de desarrollo. Por ello, estas modelos no son ideales para proyectos donde los requisitos son propensos a cambiar con el tiempo, ya que no se adaptan fácilmente a cambios significativos en el alcance del proyecto. Además, la interacción con los usuarios y los stakeholders ocurre principalmente al comienzo del proceso, durante la fase de análisis de requisitos, lo que puede llevar a malentendidos o a que los requisitos no reflejen

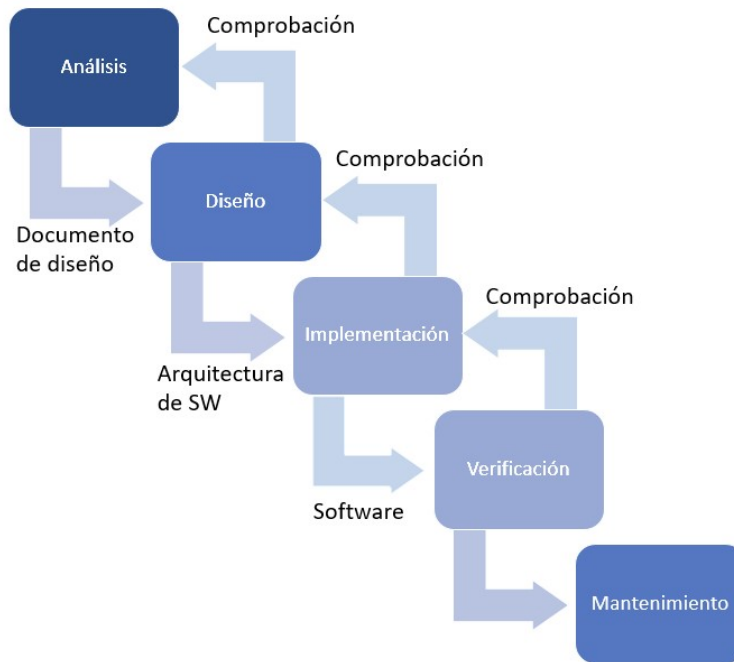


Figure 2: Modelo cascada de Winston W. Royce

completamente las necesidades del cliente. También podría llevar a un producto final que no cumpla completamente con las expectativas del cliente. Tienden a enfocarse en desarrollar todo el producto antes de entregarlo al cliente y a requerir demasiada documentación limitando el desarrollo real del software funcional y en ciertos casos, las etapas de pruebas y control de calidad pueden ser relegados al final del proceso, lo que puede dificultar la detección temprana de errores y problemas en el software. Para abordar los problemas de los modelos clásicos, muchas organizaciones han adoptado modelos ágiles, que fomentan la flexibilidad, la colaboración y la entrega incremental de valor. Estos enfoques buscan superar las limitaciones de los clásicos y adaptarse mejor a los entornos cambiantes y las necesidades del desarrollo de software en la actualidad.

2.2 Modelos ágiles

Este enfoque emergió a principios del nuevo milenio, cuando muchos desarrolladores comenzaron a cuestionar la viabilidad del modelo en cascada. Este último se mostraba insuficiente para satisfacer las necesidades cambiantes de los clientes y no proporcionaba los resultados esperados. La pérdida de recursos y tiempo impulsó la búsqueda de una alternativa más efectiva. Con la imparable transformación digital, era evidente que se requería un cambio de paradigma en el desarrollo de software.

Los modelos ágiles surgieron como respuesta a este desafío. Estos modelos se centran en crear iteraciones cortas y frecuentes del software, en las que no todas las funcionalidades están disponibles desde el principio. Aunque estas versiones pueden parecer incompletas, son de gran valor, ya que permiten obtener un feedback rápido del cliente. Esto posibilita realizar mejoras de manera cíclica, manteniendo una estrecha colaboración entre desarrolladores y clientes durante todo el ciclo de vida del proyecto.

Las prácticas ágiles, como Scrum o Kanban, se basan en la formación de pequeños equipos de trabajo que se enfocan en tareas definidas como sprints. En cuestión de semanas, se realizan ciclos de retroalimentación a través de reuniones, con el objetivo de mejorar continuamente el proceso y el producto final.

El cambio hacia los modelos ágiles ha revolucionado la forma en que se desarrolla el software, permitiendo una mayor flexibilidad, adaptabilidad y satisfacción del cliente. Al adoptar prácticas ágiles, las organizaciones pueden mantenerse alineadas con las necesidades cambiantes del mercado y

garantizar que sus productos y servicios sean altamente competitivos en la era digital.

3 Secure Software development lifecycle (S-SDLC)

Uno de los problemas más comunes en el desarrollo de software es que la seguridad se aborda en una etapa demasiado avanzada del proceso: la de pruebas, después de haber completado las tareas más importantes de diseño e implementación. En muchos casos, los controles de seguridad que se ejecutan en esa etapa son superficiales, es decir, se limitan al análisis y las pruebas de intrusión. Por eso es posible que se pasen por alto problemas de seguridad más complejos que, de detectarse, podrían retrasar la llegada del sistema a la producción. Además, la resolución de los problemas lleva mucho tiempo y es más costosa, ya que puede requerir que se vuelva a desarrollar y probar todo el software.

En la Figura 3 Se aprecia que los costes son exponenciales según avanzamos en las fases hasta alcanzar el producto final y se disparan cuando el producto está en el mercado o entregado al cliente final, y los defectos encontrados en la fase de pruebas son ya muy numerosos.

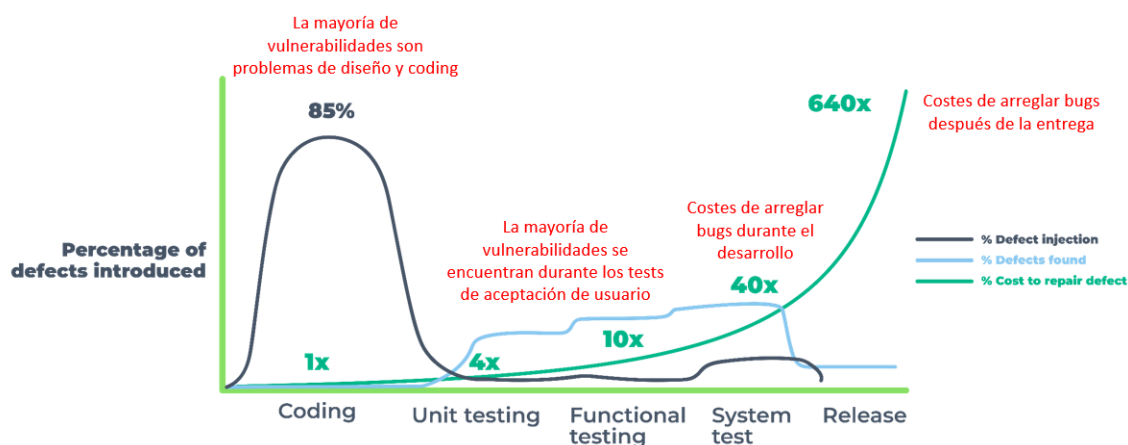


Figure 3: Porcentaje de defectos introducidos y su coste en las diferentes fases del ciclo de vida del SW [3]

S-SDLC es un conjunto de principios de diseño y buenas prácticas a implantar en el SDLC, para detectar, prevenir y corregir los defectos de seguridad en el desarrollo y adquisición de aplicaciones, de forma que se obtenga software de confianza y robusto frente a ataques maliciosos, que realice solo las funciones para las que fue diseñado, que esté libre de vulnerabilidades, ya sean intencionalmente diseñadas o accidentalmente insertadas durante su ciclo de vida y se asegure su integridad, disponibilidad y confidencialidad. Hay ciertos pasos que se pueden seguir en cada etapa para lograr el ciclo de vida de desarrollo del software seguro (SSDLC). Por ejemplo:

Table 1: SDLC Medidas de seguridad

Etapa del SDLC	Medida de Seguridad
Planificación	• Evaluar los riesgos y el panorama de las amenazas a la seguridad • Evaluar el posible impacto de los incidentes de seguridad, como el riesgo para la reputación de la empresa
Análisis de los requisitos	• Incluir los requisitos de seguridad • Comprender e incorporar los requisitos de cumplimiento normativo

Continued on next page

Table 1: SDLC Medidas de seguridad (Continued)

Diseño	• Elaborar modelos de amenazas • Incluir las consideraciones de seguridad como parte integral del plana de arquitectura • Evaluar el impacto que tiene la seguridad en las decisiones de la etapa de diseño, como la plataforma y la interfaz de usuario
Desarrollo	• Capacitar a los desarrolladores sobre las prácticas de codificación seguras • incorporar herramientas de pruebas de la seguridad • Evaluar las dependencias del software y reducir los riesgos de seguridad
Documentación	• Documentar los procesos y los controles de seguridad • Reunir la información para prepararse para las auditorías, los controles de cumplimiento normativo y las revisiones de seguridad
Pruebas	• Implementar procesos de revisión del código • Realizar pruebas estáticas y dinámicas de la seguridad de las aplicaciones
Implementación	• Evaluar la seguridad del entorno de implementación • Revisar las configuraciones de seguridad
Mantenimiento	• Supervisar el sistema para detectar amenazas • Prepararse para eliminar puntos vulnerables y solucionar las intrusiones

3.1 Microsoft Security Development Lifecycle (SDL)

Microsoft Security Development Lifecycle (SDL) [2] es un proceso propuesto por Microsoft en 2004, diseñado para mejorar la seguridad del desarrollo de software. Consta de siete componentes, incluidas cinco fases principales y dos actividades de seguridad auxiliares. Cada una de estas fases contiene comprobaciones y aprobaciones obligatorias para garantizar que todos los requisitos de seguridad y privacidad y los procedimientos recomendados se aborden correctamente.

1. Formación: En esta área, se destaca la importancia de proporcionar una formación continua a los roles técnicos dentro del equipo de desarrollo. El objetivo es asegurar que el equipo esté actualizado con las últimas prácticas y conocimientos de seguridad, permitiéndoles responder adecuadamente a los cambios tecnológicos y amenazas emergentes.
2. Requisitos y diseño: Esta área se enfoca en la identificación y documentación de los requisitos de seguridad del software, así como en la aplicación de técnicas de diseño seguro. Es esencial definir desde el principio los aspectos de seguridad que se deben cumplir en el producto final. El modelo de amenazas es una característica fundamental del SDL, apoyado por la threat modelling tool de Microsoft. Ayuda a los desarrolladores a identificar áreas potencialmente vulnerables del código, permitiéndoles enfocar sus esfuerzos en fortalecer esas áreas y mitigar riesgos.
3. Implementación: Aquí se centra en la codificación segura mediante herramientas de desarrollo seguras para implementar de forma eficaz todos los requisitos de seguridad, privacidad y función del software que diseñan. También se le da importancia a la aplicación de buenas prácticas de programación para evitar errores comunes que puedan llevar a vulnerabilidades de seguridad. Se enfatiza en el análisis estático del código para detectar posibles problemas durante la fase de desarrollo.
4. Comprobación: La comprobación de seguridad es crucial para identificar y corregir vulnerabilidades antes del lanzamiento del software. El análisis dinámico, el fuzzing y otras técnicas de prueba se utilizan para validar la seguridad del producto.
5. Lanzamiento y respuesta: Una vez que el software ha pasado por las etapas anteriores, se implementan medidas para asegurar su lanzamiento seguro. Para ello, las compilaciones se publican sistemática y gradualmente en grupos más grandes y grandes, lo que se conoce como anillos, en lo que se denomina proceso de implementación segura (SDP). Además, se planifica una respuesta efectiva ante posibles incidentes de seguridad que puedan surgir en el futuro.

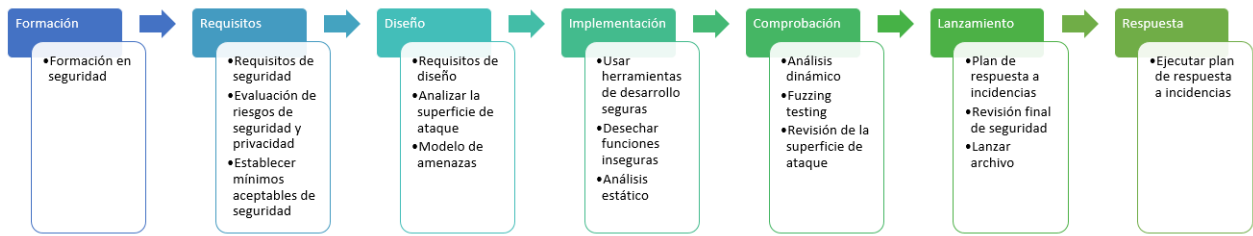


Figure 4: Microsoft SDL

3.2 Proceso de seguridad en aplicación completo y ligero (CLASP)

CLASP (Comprehensive Lightweight Application Security Process) es un proyecto de OWASP que establece una serie de actividades, roles y buenas prácticas dirigidas a coordinar los procesos de desarrollo seguro de software. La organización OWASP CLASP se asienta en cinco perspectivas o vistas (Figura 5) que abordan los conceptos generales de esta metodología, la distribución de funciones, la valoración de las actividades aplicables, la implementación de estas actividades, y el listado de problemas que pueden dar lugar a la aparición de vulnerabilidades:

1. Vista de conceptos: Esta vista proporciona una introducción de alto nivel a la metodología CLASP. Describe de manera concisa la interacción de las diferentes vistas, las mejores prácticas y la secuencia en la que se deben aplicar los componentes.
2. Vista basada en roles: En esta etapa, se presentan las instrucciones del proceso basadas en las funciones de los miembros del equipo. Esto ayuda a los participantes a comprender sus responsabilidades específicas en relación con la seguridad durante el desarrollo del software.
3. Vista de evaluación de actividades: Esta vista asiste a los responsables del proyecto en la evaluación y selección de las actividades más apropiadas para aplicar durante el desarrollo. CLASP proporciona cierto soporte y orientación para tomar decisiones informadas en esta área.
4. Vista de implementación de actividades: Aquí se detallan todas las actividades de seguridad de CLASP que pueden integrarse en el proceso de desarrollo de software. Estas actividades, evaluadas y aceptadas durante la evaluación del SDLC, contribuyen a generar un software seguro y resistente a amenazas.
5. Vista de vulnerabilidades: Esta vista comprende un catálogo de diferentes problemas previamente identificados por CLASP, que conforman una base de datos de vulnerabilidades que podrían surgir en el código fuente del software. Estas vulnerabilidades se clasifican en cinco categorías distintas. Además, se describen las condiciones en las que los servicios de seguridad suelen tener vulnerabilidades en la capa de aplicación. Se proporcionan ejemplos prácticos y sencillos para entender la causa de las fallas y sus posibles soluciones.

CLASP identifica 104 tipos de problemas subyacentes que forman la base de la seguridad de vulnerabilidades en el código fuente de la aplicación. Un tipo de problema individual en sí mismo, a menudo no es una vulnerabilidad de seguridad, con frecuencia es una combinación de problemas que crean una falsa condición de seguridad que conduce a una vulnerabilidad en el código fuente.

Los 104 tipos de problemas se clasifican según CLASP en 5 categorías de alto nivel, donde cada tipo de problema puede tener más de una categoría matriz: errores de rango y tipo, problemas de entorno, errores de tiempo y sincronización, errores de protocolo y errores lógicos generales. CLASP define una consecuencia de una vulnerabilidad explotada o explotable como un fallo en uno o más de estos servicios básicos de seguridad:

- Autorización (control de acceso a recursos)
- Confidencialidad (de datos u otros recursos)
- Autenticación (establecimiento e integridad de la identidad)

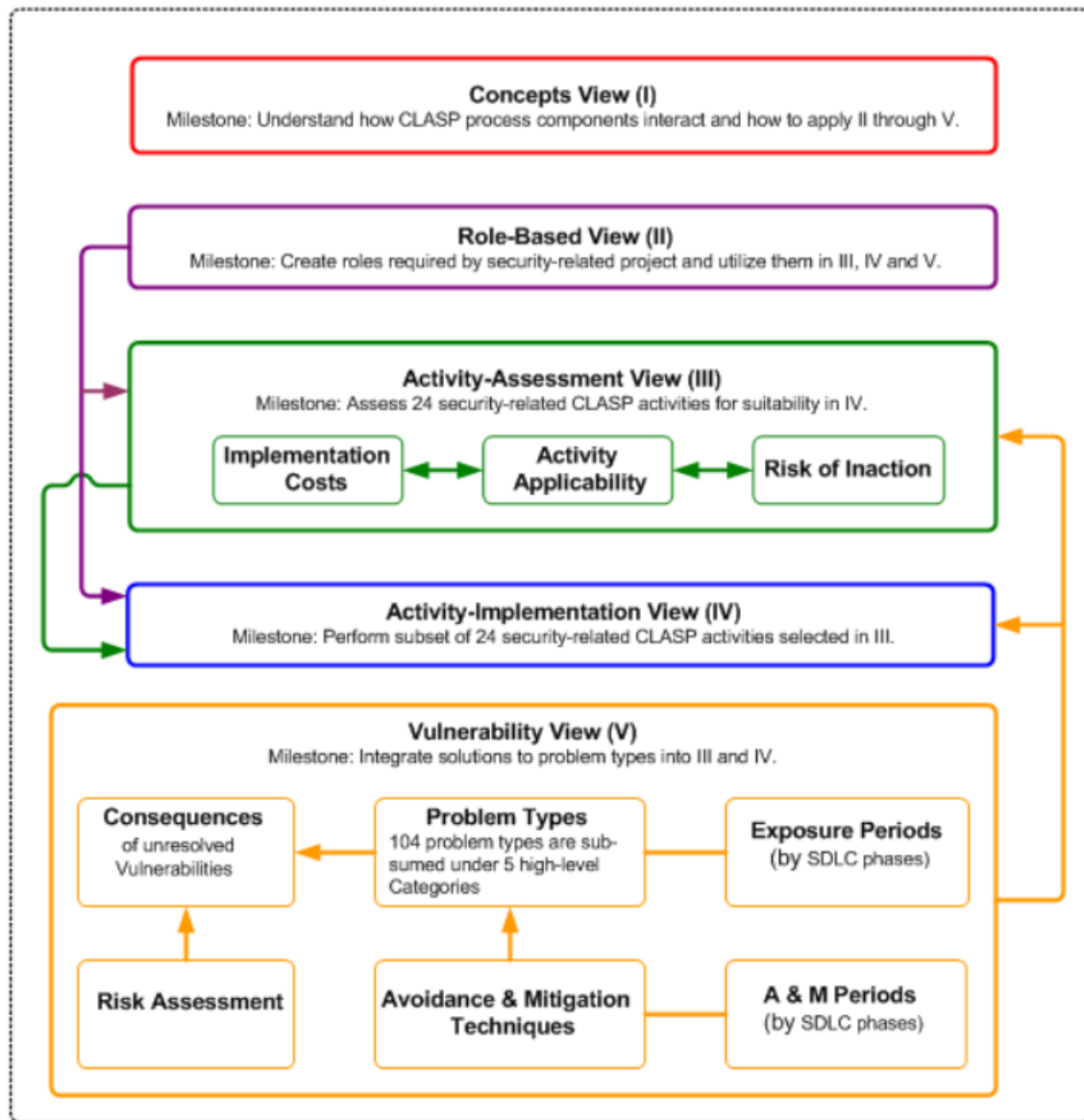


Figure 5: Vistas CLASP [6]

- Disponibilidad (negación de servicio)
- Responsabilidad
- No repudio

Otro punto para tener en cuenta son las mejores prácticas que promueve el marco de CLASP:

- Instituir programas de concienciación: Los conceptos y técnicas esenciales de seguridad pueden ser ajenos a los desarrolladores de software de la organización y otros involucrados en la aplicación del desarrollo y despliegue. Por lo tanto, es imperativo desde el principio educar a todos los involucrados. Es fundamental, que los gerentes de proyecto, como fuerza impulsora detrás de la mayoría de los proyectos de actualización o desarrollo de aplicaciones, tengan en cuenta y promuevan que la seguridad ha de ser un objetivo importante del proyecto, tanto a través de la capacitación como de la rendición de cuentas. Los programas de concienciación se pueden implementar fácilmente, utilizando expertos externos o recursos, según corresponda, y generar

un alto rendimiento al ayudar a garantizar que otras actividades que promuevan el software seguro se implementarán de manera efectiva.

- Realizar evaluaciones de aplicaciones. La aplicación de pruebas y evaluaciones deben seguir siendo un componente central de la estrategia de seguridad. Las evaluaciones, en particular las pruebas automatizadas, pueden encontrar problemas de seguridad no detectados durante las revisiones de código o implementación. Es muy importante encontrar riesgos de seguridad introducidos por el entorno operativo, y actuar en consecuencia como un mecanismo de defensa en profundidad al detectar errores en el diseño, especificación o implementación.
- Capturar requisitos de seguridad: Asegurar que los requisitos de seguridad tengan el mismo nivel de importancia que todos otros “imprescindibles”. Es fácil para los arquitectos de aplicaciones y los administradores de proyectos centrarse en la funcionalidad al definir los requisitos, ya que soportan el mayor propósito de la aplicación para entregar valor a la organización. Las consideraciones de seguridad pueden pasar fácilmente por el camino. Por lo tanto, es crucial que la seguridad en los requisitos sea una parte explícita de cualquier esfuerzo de desarrollo de aplicaciones. Es necesario considerar cómo se utilizarán las aplicaciones y cómo podrían ser un ataque dirigido o malintencionado, los activos (datos y servicios) a los que accederá o proporcionará la aplicación, y qué nivel de protección es apropiado dadas las necesidades de su organización, las regulaciones a las que está sujeto y el impacto potencial en la pérdida de reputación en caso de que una aplicación sea explotada. La arquitectura de la aplicación y probables vectores de ataque y los controles compensatorios potenciales, y su costo y efectividad.
- Implementar prácticas de desarrollo seguras: Actividades de seguridad definidas, artefactos, alineamientos y refuerzo continuo deben convertirse en parte de la cultura general de la organización.
- Construir procedimientos de remediación de vulnerabilidades: Es especialmente importante en el contexto de las actualizaciones de aplicaciones y mejoras para definir qué pasos se tomarán para identificar, evaluar, priorizar y remediar vulnerabilidades.
- Definir y monitorear métricas: No se puede gestionar lo que no se puede medir. Desafortunadamente, implementar un esfuerzo efectivo de monitoreo de métricas puede ser una tarea difícil. A pesar de ello, en este sentido, las métricas son un elemento esencial de su esfuerzo general de seguridad de aplicaciones. Son cruciales para evaluar la postura de seguridad actual de una organización, para ayudar a centrar la atención en las vulnerabilidades más críticas y revelar qué tan bien o mal están funcionando las inversiones en mejoras de seguridad.
- Publicar alineamientos de seguridad operacional. La seguridad no termina cuando una aplicación se completa y se implementa en un entorno de producción. Hay que aprovechar al máximo la red existente y las inversiones en seguridad operativa que requieren que se informe y se eduque a aquellos encargados de monitorear y administrar la seguridad de los sistemas en ejecución con asesoramiento y orientación sobre los requisitos de seguridad que exige la aplicación, junto a la mejor manera de hacer uso de las capacidades que se han integrado en el aplicativo

3.3 Secure Software Development Framework (SSDF)

El Instituto Nacional de Estándares y Tecnología (NIST) desarrolló el marco de Desarrollo de Software Seguro (SSDF) [7] con el objetivo de mejorar la resistencia a las vulnerabilidades en el software. En lugar de introducir terminología nueva, el SSDF consolida las mejores prácticas y pautas existentes para el desarrollo de software seguro establecidas por organizaciones como BSA, OWASP o SAFECODE. Se enfoca en las prácticas óptimas, sin limitarse a herramientas, técnicas o mecanismos específicos para implementarlas. Proporciona una guía integral y estructurada para mejorar la seguridad del desarrollo de software en todas las etapas.

Como se describe en la Figura 6, SSDF se divide en cuatro categorías principales de prácticas, cada una abordando un aspecto esencial del desarrollo de software seguro:

1. Preparar la Organización (PO): En esta categoría, se enfatiza la importancia de asegurar que la organización esté preparada para implementar medidas de seguridad en todo el proceso de desarrollo de software. Esto incluye asegurarse de que los procesos, las personas y la tecnología

dentro de la organización estén listos para garantizar la seguridad a nivel organizacional. También se proporcionan prácticas específicas que pueden aplicarse a subconjuntos de la organización, como grupos de desarrollo individuales o proyectos específicos.

2. Proteger el Software (PS): Aquí, el enfoque se centra en garantizar que todos los componentes de software estén protegidos contra el acceso no autorizado y la manipulación por parte de actores maliciosos. Se recomienda la implementación de medidas sólidas de seguridad para proteger el software desde su diseño hasta su implementación.
3. Producir Software Seguro (PW): Esta categoría hace hincapié en la necesidad de producir software que esté bien protegido desde la cadena de suministro y que cuente con medidas mínimas de seguridad para abordar vulnerabilidades potenciales. También se considera la gestión adecuada de vulnerabilidades conocidas en la solución.
4. Responder a las vulnerabilidades: Esta categoría se centra en la capacidad de las organizaciones para identificar y corregir cualquier vulnerabilidad restante en las versiones de software. Esto incluye establecer mecanismos de respuesta rápida y efectiva ante posibles amenazas emergentes.

Algunas de sus prácticas más concretas son:

- Definir criterios para las comprobaciones de seguridad del software.
- Proteger todas las formas de código contra el acceso no autorizado y la manipulación, salvaguardando los entornos de desarrollo, construcción, distribución y actualización y siguiendo el principio de mínimo privilegio.
- Proporcionar un mecanismo para verificar la integridad de la versión del software mediante la firma digital del código a lo largo del ciclo de vida del software.
- Diseñar el software para cumplir con los requisitos de seguridad y mitigar los riesgos de seguridad.
- Verificar que el software de terceros cumple con los requisitos de seguridad.
- Configurar los procesos de compilación y construcción para mejorar la seguridad de los ejecutables.
- Revisar y/o analizar el código legible por el ser humano para identificar vulnerabilidades y verificar el cumplimiento de los requisitos de seguridad.
- Probar el código ejecutable para identificar vulnerabilidades y verificar el cumplimiento de los requisitos de seguridad.
- Configurar el software para que tenga una configuración segura por defecto.
- Archivar y proteger cada versión del software.
- Identificar, analizar y corregir las vulnerabilidades de forma continua.

En la Figura 7 se puede ver un ejemplo concreto de prácticas, tareas y ejemplos de implementación para la categoría PS [7].

4 Implementación del SSDLC con DevSecOps

4.1 DevOps

Esta filosofía se basa en que las personas trabajen juntas para concebir, construir y entregar software seguro a alta velocidad. Las prácticas de DevOps permiten a los equipos de desarrollo de software (dev) y operaciones (ops) acelerar la entrega a través de la automatización, la colaboración, la retroalimentación rápida y la mejora iterativa. Comparte una estrecha relación con el desarrollo ágil, para obtener como resultado un método colaborativo entre el desarrollo y las operaciones. Tiene como objetivo integrar el trabajo de los equipos de desarrollo y los equipos de operaciones al facilitar una cultura de colaboración y responsabilidad compartida. Al adoptar un proceso de desarrollo de DevOps, se

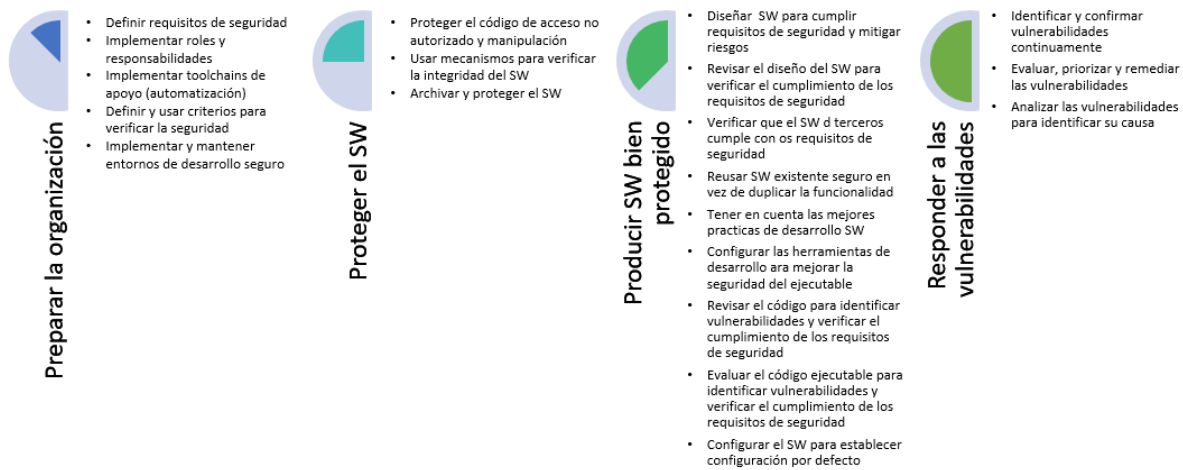


Figure 6: SSDF

Practices	Tasks	Notional Implementation Examples
Provide a Mechanism for Verifying Software Release Integrity (PS.2): Help software acquirers ensure that the software they acquire is legitimate and has not been tampered with.	PS.2.1: Make software integrity verification information available to software acquirers.	Example 1: Post cryptographic hashes for release files on a well-secured website. Example 2: Use an established certificate authority for code signing so that consumers' operating systems or other tools and services can confirm the validity of signatures before use. Example 3: Periodically review the code signing processes, including certificate renewal, rotation, revocation, and protection.
Archive and Protect Each Software Release (PS.3): Preserve software releases in order to help identify, analyze, and eliminate vulnerabilities discovered in the software after release.	PS.3.1: Securely archive the necessary files and supporting data (e.g., integrity verification information, provenance data) to be retained for each software release.	Example 1: Store the release files, associated images, etc. in repositories following the organization's established policy. Allow read-only access to them by necessary personnel and no access by anyone else. Example 2: Store and protect release integrity verification information and provenance data, such as by keeping it in a separate location from the release files or by signing the data.
	PS.3.2: Collect, safeguard, maintain, and share provenance data for all components of each software release (e.g., in a software bill of materials [SBOM]).	Example 1: Make the provenance data available to software acquirers in accordance with the organization's policies, preferably using standards-based formats. Example 2: Make the provenance data available to the organization's operations and response teams to aid them in mitigating software vulnerabilities. Example 3: Protect the integrity of provenance data, and provide a way for recipients to verify provenance data integrity. Example 4: Update the provenance data every time any of the software's components are updated.

Figure 7: Ejemplos de prácticas en SSDF



Figure 8: DevOps [1]

toma la decisión de mejorar el flujo y la entrega de valor de su aplicación al fomentar un ambiente de colaboración en todas las etapas del ciclo de desarrollo.

DevOps representa sobre todo un cambio en la mentalidad de la cultura SW, enfocándose en el desarrollo incremental y la entrega rápida de software. La comunicación y la confianza entre el equipo se vuelve indispensable. Para poder realizar una entrega óptima para pasar a producción, los desarrolladores necesitan transparencia en todos los pasos. Esto lleva a extender la colaboración con otros equipos, como calidad, infraestructura o seguridad. El éxito radica precisamente en la capacidad de crear una cultura de responsabilidad, colaboración mejorada, empatía y responsabilidad conjunta por los resultados del negocio.

DevOps se basa en crear un desarrollo optimizado para obtener resultados gracias a los feedbacks más inmediatos. La responsabilidad de entregar el software a los clientes recae en todo el equipo, por lo que se obtiene una visibilidad total para conocer cómo se utiliza el software y solucionar cualquier problema planteado. Además, automatizar ciertos métodos es indispensable en DevOps, para mejorar la calidad y no perder tiempo con elementos que serán descartados, gracias al rápido feedback que se genera.

4.2 DevSecOps

Tradicionalmente, lo usual era preocuparse por la seguridad de las aplicaciones solo después de completar todo el desarrollo. El equipo de seguridad trabajaba separado del equipo de desarrollo y operaciones, lo que ralentizaba los procesos y la solución de problemas. Además, las herramientas de seguridad se usaban de forma aislada, enfocándose solo en aplicaciones específicas y sin tener una visión general de los problemas de seguridad que podían surgir. Esto limitaba el conocimiento de los riesgos en el software al equipo de seguridad, dejando a otros miembros del equipo sin una comprensión completa de los riesgos en el entorno de producción.

DevSecOps, también conocida como SecDevOps, es una filosofía de desarrollo de software que promueve la adopción de la seguridad en todo el ciclo de vida del desarrollo de software (SSDLC). DevSecOps va más allá de ser una herramienta o una práctica en concreto; nace como evolución de la metodología DevOps y su principal motivación es automatizar la seguridad para responder a la aceleración en los ciclos de lanzamiento de software promovida por la adopción de DevOps. DevSecOps no solo añade elementos de seguridad a ciclos DevOps, sino que aplicada correctamente, hace de la seguridad una parte integral dentro de todo el proceso, desde el inicio hasta el final.



Figure 9: Dev & Sec & Ops

Cuando la seguridad de una aplicación se integra en el proceso DevSecOps, -desde los inicios hasta su salida a producción-, las organizaciones tienen la capacidad de implementar de manera conjunta los tres componentes más importantes de la creación y entrega de software. De igual manera, permite realizar las pruebas de seguridad de una manera transparente, automática y en paralelo a otros desarrollos

o pruebas. Por ejemplo, se pueden ejecutar pruebas de seguridad durante el propio desarrollo de la aplicación, sin producir contratiempos al trabajo de otros equipos. De la misma manera, se pueden ejecutar pruebas durante la fase de producción, con la intención de encontrar vulnerabilidades para corregirlas poco después de ser localizadas.

Como consecuencia, el equipo de seguridad se compromete mucho más con el resto de equipos que intervienen en el SDLC, incluyendo Desarrollo y Operaciones. Esto elimina fricción, ya que la tensión natural que existe entre velocidad y seguridad se comparte por todos los equipos.

Mientras que el Secure SDLC involucra mucho más que aquello que se puede automatizar, por ejemplo, las capacitaciones o entrenamientos, modelado de amenazas, revisiones de diseño o arquitectura de software, para DevSecOps es casi indispensable que se trate de procesos que se pueden automatizar e incluir dentro de "tuberías de entrega" (Delivery pipeline) con soluciones de Integración y Entrega continuos (CI/CD).

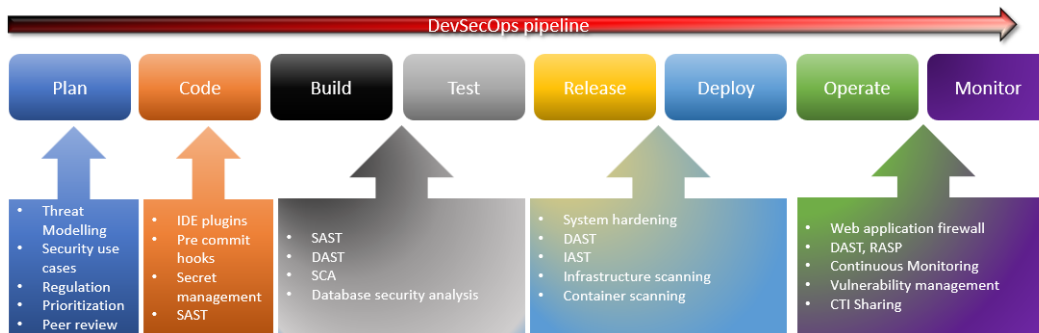


Figure 10: DevSecOps - Actividades de ciberseguridad [4]

Veamos paso a paso qué actividades podemos realizar durante el ciclo de vida de nuestro producto para garantizar su ciberseguridad. No es obligatorio agregar todos estos pasos, pero cada uno ayuda a revelar problemas de seguridad cuanto antes, cuando son mucho más baratos y más fáciles de corregir.

4.2.1 Planificación

Normalmente, el desarrollo moderno sigue una metodología de desarrollo ágil. Scrum es una implementación con una metodología ágil en la que cada sprint se inicia con una actividad de planeamiento. La introducción de la seguridad en esta parte del proceso de desarrollo debe centrarse en lo siguiente [5]:

- Modelado de amenazas para ver la aplicación desde la perspectiva de un posible atacante, identificar como una atacante puede abusar de la aplicación y cómo podemos evitarlo. Hay herramientas que ayudan a automatizar esta parte como Microsoft Threat Modeling Tool, OWASP Threat Dragon, IriusRisk, OWASP pytm, ThreatModeler o ThreatPlayBook. Además, hay herramientas lúdicas de formación para aprender a modelar amenazas como el juego de Microsoft Elevation of Privilege (EoP)¹.
- Identificación de directrices de prácticas de codificación, como la guía rápida de OWASP², estándares de seguridad según el contexto (e.g. estándares de IoT y Cloud) y regulaciones que sean aplicables a nuestro proyecto, como el Reglamento General de Protección de Datos (RGPD), la Ley Orgánica de Protección de Datos y Garantía de los Derechos Digitales (LOPDGDD) o la Ley de Servicios de la Sociedad de la Información y el Comercio Electrónico (LSSI).
- Definición de casos de uso y requisitos teniendo en cuenta la ciberseguridad. Hay recursos educativos como el juego OWASP Cornucopia³ para aprender a identificar requisitos de seguridad.
- Priorización. En ciberseguridad es imposible garantizar que algo es 100% seguro, hay que valorar el tiempo que queremos dedicar a proteger nuestro código y los beneficios que obtendremos. Por

¹<https://www.microsoft.com/en-gb/download/details.aspx?id=20303>

²<https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/>

³<https://owasp.org/www-project-cornucopia/>

tanto, es importante identificar que problemas son más urgentes o pueden hacer más daño al producto.

- Revisión por pares. Parte del proceso de solicitud de incorporación de cambios son las revisiones por pares que suelen detectar defectos, errores o problemas no identificados relacionados con errores humanos. Es un procedimiento recomendado tener un compañero del equipo de seguridad experto o un experto en seguridad que pueda guiar al desarrollador durante el proceso de revisión por pares antes de crear una solicitud de incorporación de cambios

Las fases de planificación de DevSecOps son las menos automatizadas, ya que involucra colaboración y discusión para que los equipos realicen un análisis de seguridad. Sin embargo, también existen herramientas para la colaboración y la conversación, como Slack, así como soluciones para gestionar y hacer seguimiento de problemas, como Jira Software.

4.2.2 Desarrollo/Codificación

Esta fase implica el diseño del software y la creación del código. Normalmente, los desarrolladores se centran en la velocidad de entrega, y los controles de seguridad pueden ralentizar el proceso. Para acelerar el proceso y proporcionar comentarios inmediatos, merece la pena agregar pasos, como:

- Pruebas de seguridad de aplicaciones estáticas (SAST): escanean el código (cuando la aplicación no se está ejecutando) en busca de debilidades y problemas de seguridad que deben abordarse. Hay muchas herramientas gratuitas y comerciales en este ámbito⁴ pero a menudo generan muchos falsos positivos y no se pueden aplicar a todos los lenguajes de programación. Además, suelen tardar horas (o incluso días) en ejecutarse, por lo que la mejor práctica es realizar pruebas de código incrementales durante los días laborables y escanear todo el código durante el fin de semana. Podemos destacar SonarCloud, que incluye varias herramientas SAST para distintos lenguajes, SonarQube que identifica los puntos susceptibles de mejora para la optimización del código y Gitleaks para detectar y prevenir secretos codificados como contraseñas, claves de API y tokens en repositorios de Git.
- Complementos de seguridad del entorno de desarrollo integrado (IDE): identifican diferentes problemas de seguridad durante el proceso de desarrollo en el IDE con el que está familiarizado. Los complementos pueden proporcionar comentarios inmediatos si hay un riesgo de seguridad potencial en el código escrito del desarrollador e incluso revelar riesgos en la biblioteca o el paquete de terceros. Según el IDE que elija, muchas empresas de seguridad proporcionan muchos complementos comerciales o de código abierto. Algunos complementos son DevSkim, para encontrar errores y aplicar estándares de código, JFrog, que permite ver paneles que muestran información sobre vulnerabilidades en los componentes y sus dependencias directamente en Eclipse, o Snyk, que ofrece escaneo de vulnerabilidades.
- Precommit hooks: las herramientas de esta categoría ayudan a identificar problemas antes de que el desarrollador envíe el código para su revisión. Evitan por ejemplo, que se incluya información sensible, como credenciales, en la plataforma de gestión de código. Hay opciones de código abierto disponibles, como git-hound, git-secrets, Precommit hook GitHub y repo-supervisor.
- Gestión de secretos: con la automatización, almacenar credenciales en archivos de configuración y variables de entorno para acceder a varios servicios es una práctica habitual seguida por desarrolladores y administradores. No obstante, almacenar credenciales en archivos o configuraciones puede llevar a exponer credenciales a una audiencia inadecuada. Servicios de gestión de secreto como Hashicorp ayudan a gestionar el acceso a credenciales sensibles en entornos de baja confianza. Los secretos se definen como cualquier forma de credencial sensible que debe ser estrictamente controlada y supervisada y que puede ser utilizada para desbloquear información sensible, como contraseñas, claves de encriptación de la API, etc. Hashicorp permite tanto almacenar secretos como generar dinámicamente el acceso a servicios y aplicaciones específicas a través de su API e incluso crear registros para hacer un seguimiento de quién ha accedido a qué. Empresas como AWS, Microsoft y Google también tienen sus propias soluciones.

⁴https://owasp.org/www-community/Source_Code_Analysis_Tools

4.2.3 Compilación y prueba

En esta fase se gestionan las versiones y las compilaciones del software, y se utilizan herramientas automatizadas que ayudan a compilar y crear paquetes de código para publicarlos después para la producción. Una vez compilado, deben realizarse pruebas para garantizar la seguridad de la programación. Algunas de las actividades en estas fases incluyen:

- Pruebas SAST (caja blanca)
- Pruebas de Seguridad de Aplicaciones Dinámicas (DAST, caja negra): detectan problemas de seguridad en la aplicación en ejecución al observar cómo responde la aplicación a solicitudes diseñadas específicamente. Proporciona menos falsos positivos que SAST, pero igualmente consume mucho tiempo. Algunos ejemplos de herramientas DAST son OWASP ZAP, arachni scanner, Subgraph Vega, Skipfish, Nikto, GoLismero o Wapiti.
- Análisis de seguridad de bases de datos: verifican parches y versiones actualizadas, contraseñas débiles, errores de configuración, problemas de lista de control de acceso (ACL), etc. Un ejemplo es Oracle Database Security Assessment.
- Herramientas de Análisis de Composición de Software (SCA): examinan las bibliotecas que se utilizan en el proyecto y señalan las que tienen vulnerabilidades para detectar SW malicioso o vulnerable de terceros. Para ello, inspeccionan administradores de paquetes, archivos de manifiesto, código fuente, archivos binarios, imágenes de contenedores, etc. y crean una lista de los componentes (Lista de materiales (BOM)), centrándose en los de código abierto. Algunas de estas herramientas son OWASP Dependency-Check, OWASP Dependency-Track, Bundler Audit, Retire JS o Rubysec.

4.2.4 Producción y Despliegue

En esta fase se gestionan, coordinan, programan y automatizan las tareas de producción de las versiones de productos. Algunas de las actividades en estas fases incluyen:

- Pruebas DAST (caja negra)
- Pruebas de Seguridad de Aplicaciones Interactivas (IAST): combinan elementos de SAST y DAST al colocar un agente dentro de la aplicación y realizar un análisis en tiempo real en cualquier momento del proceso de desarrollo. Como resultado, la prueba abarca tanto el código fuente como todos los demás elementos externos, como bibliotecas y APIs (esto no era posible con SAST o DAST, por lo que los resultados son más precisos). Sin embargo, este tipo de pruebas puede tener un impacto adverso en el rendimiento de la aplicación.
- System hardening: el endurecimiento es el proceso de asegurar un sistema reduciendo su superficie de ataque. En principio un sistema con una única función es más seguro que uno con muchos propósitos. El proceso de cerrar las vías para los ataques más típicos incluye el cambio de claves por defecto, desinstalar el software y dar de baja usuarios y accesos innecesarios; también deshabilitar servicios que no serán usados y fortalecer las configuraciones de aquellos que estarán en uso. Existen guías de endurecimiento y herramientas como Lynis, Bastille Linux, JASS para sistemas Solaris y Apache/PHP Hardener, que pueden, por ejemplo, desactivar características innecesarias en los archivos de configuración o actuar de diversas otras formas para proporcionar protección.
- Escaneo de imágenes y contenedores en la nube: verifican el código fuente y su cumplimiento con las mejores prácticas de configuraciones seguras. A medida que los contenedores ganan popularidad, se convierten en un objeto de interés para los productores de malware. Por lo tanto, es necesario escanear las imágenes de Docker que se descargan de repositorios públicos, y herramientas como Clair facilitan este proceso resaltando posibles vulnerabilidades.
- Escaneo de infraestructura: detectan y corrigen automáticamente vulnerabilidades de seguridad y problemas de configuración en entornos en la nube. Tecnologías como Microsoft Defender for Cloud y Microsoft Sentinel son herramientas nativas que se integran en los entornos de Azure.

4.2.5 Operaciones y monitorización

En operaciones se gestiona el software durante su producción y se recopila información sobre problemas que surgen en una versión de software específica que se encuentra en producción. Durante la monitorización se detectan y notifican cualquier evento o configuración anómala que requiere una investigación y una posible corrección. Algunas de las actividades en estas fases incluyen:

- Pruebas DAST (caja negra)
- Web Application Firewall (WAF) permite definir reglas de red específicas para una aplicación web y filtrar, supervisar y bloquear el tráfico HTTP hacia y desde un servicio web cuando corresponde a patrones conocidos de ataques, como la inyección de SQL. Todos los principales proveedores de servicios en la nube como Google, AWS y Microsoft tienen su propio WAF, pero también existen empresas especializadas como Cloudflare, Imperva y Wallarm, entre otras.
- Autoprotección de aplicaciones en tiempo de ejecución (RASP) permite que las aplicaciones realicen controles continuos de seguridad y reaccionen a los ataques en tiempo real al eliminar al atacante (por ejemplo, cerrando su sesión) y alertando al equipo. Al igual que IAST, puede afectar el rendimiento de la aplicación. En comparación con el enfoque perimetral WAF, un RASP protege las aplicaciones desde el interior. Esto significa que un RASP tiene una visibilidad mucho mejor del flujo de datos y las consecuencias de cada entrada que recibe la aplicación.
- Herramientas de monitorización: la monitorización es una parte crucial de DevOps. DevSecOps lo lleva al siguiente nivel y abarca no solo aspectos como el tiempo de inactividad, sino también amenazas de seguridad. En esta categoría podemos encontrar herramientas de detección de intrusiones (IDS) como Snort o herramientas de Gestión de Información y Eventos de Seguridad (SIEM) que proporcionan una correlación de la información de seguridad supervisada desde diferentes fuentes como por ejemplo Elastic entre otras⁵.
- Gestión de vulnerabilidades: medidas tomadas para detectar y discernir, clasificar, priorizar y luego resolver las vulnerabilidades. Este proceso no es estático, sino un esfuerzo en tiempo real para detectar y resolver constantemente estas vulnerabilidades. Se hace mediante una combinación de parches consistentes y reconfiguración de controles de seguridad. Todo comienza con la identificación de los puntos débiles dentro de los activos digitales y la recopilación de la mayor cantidad de información posible. Esto incluye la identificación de vulnerabilidades (escáneres de vulnerabilidades), evaluación (CVSS, CVE), tratamiento y generación de informes. Estas herramientas ayudan a identificar las debilidades en los sistemas de seguridad y clasificar las vulnerabilidades según el impacto potencial de ataques maliciosos que se aprovechen de ellas, para que podamos centrarnos en corregir las más peligrosas. Algunas de las herramientas pueden incluir complementos que solucionan automáticamente los errores encontrados. Esta categoría está llena de soluciones de código abierto⁶. Se pueden combinar con otras herramientas de respuesta a incidentes como TheHive y de compartición de información de amenazas.
- Compartición de información de amenazas (CTI sharing): CTI, Cyber threat intelligence es a lo que llamamos a la información de ciberamenazas una vez ha sido recopilada, evaluada y analizada. Compartir la información identificada es crucial para poder abordar los ataques lo antes posible. Existen diferentes modelos de intercambio de CTI, dependiendo del tipo, nivel y alcance de la información, así como de la confianza y la relación entre los participantes. Para facilitar la colaboración y el intercambio de CTI, es esencial utilizar estándares y formatos comunes que permitan la interoperabilidad, consistencia y calidad de la información. La expresión estructurada de información sobre amenazas (STIX) es un lenguaje estandarizado para expresar e intercambiar CTI de una manera legible por máquina y por humanos. STIX define diferentes objetos relacionados con CTI, como indicadores, tácticas, técnicas, procedimientos, campañas, incidentes y actores de amenazas. Además, el intercambio automatizado confiable de información de inteligencia (TAXII) es un protocolo estandarizado para transportar e intercambiar CTI a través de Internet. TAXII define métodos, servicios y mensajes para solicitar, enviar, recibir y suscribirse

⁵<https://www.gartner.com/reviews/market/security-information-event-management>

⁶<https://awesomeopensource.com/projects/vulnerability-management>

a CTI utilizando STIX. La expresión de observables cibernéticos (CybOX) es otro lenguaje estandarizado para describir y caracterizar los observables cibernéticos, como archivos, procesos, conexiones de red y claves de registro. CybOX se utiliza como parte de STIX para representar los elementos observables de CTI. Para facilitar la colaboración y el intercambio de CTI, existen varias plataformas y herramientas disponibles, como por ejemplo MISP⁷.

4.2.6 Cómo tener éxito al aplicar DevSecOps

Sensibilización responsabilidad de la seguridad. Todos los miembros de los diferentes equipos que participen en el desarrollo deben conocer los fundamentos de seguridad y sentirse responsable de los resultados obtenidos. De hecho, a pesar de que los miembros del equipo desempeñen un gran trabajo como DevOps, es habitual que tengan que volver a tomar formación para que adquieran los conocimientos y comprendan ciertas técnicas de seguridad, para operar de manera eficaz con las nuevas herramientas de seguridad con las que trabajarán.

Se debe animar a todos los miembros del equipo a colaborar, dando la misma importancia a la seguridad como a la velocidad de lanzamiento de un nuevo ciclo de desarrollo. Cuando asimilan la importancia que tiene su participación, serán capaces de incorporar ideas innovadoras, convenientes para afrontar los problemas de seguridad sin verse perjudicada la velocidad del desarrollo. El ideal que debe calar hondo es el de *"la seguridad es responsabilidad de todos"*, es uno de los pilares que forman las bases de la cultura DevSecOps. Deben asimilar y adoptar el papel de que son los responsables de implementar la seguridad en el software que desarrollan, a la vez que mantengan su funcionalidad y usabilidad.

Operaciones automatizadas. La mayoría del conjunto de herramientas de CI/CD cuentan con un alto grado de automatización. Las herramientas DevSecOps no pueden quedarse atrás en este aspecto. Estas deben contar con una automatización completa, donde los pasos manuales deben ser nulos o prácticamente ninguno. En todo momento se debe facilitar la información relevante sobre la seguridad de la aplicación, incluso cuando los desarrolladores quieran evitar pruebas que puedan ralentizar su trabajo.

Herramientas adecuadas. Hay que identificar qué herramientas son las más apropiadas para aplicar en cada fase y proyecto, con la finalidad de que su integración en el flujo de trabajo sea lo más natural y eficiente posible. Contra más automatizadas están las herramientas elegidas y más integradas con el CI/CD, menos formación y cambio cultural requerirá el equipo. Sin embargo, hay que tener en cuenta que las herramientas pueden no ser capaces de realizar detecciones precisas ante posibles vulnerabilidades.

Definición de políticas de seguridad para los desarrolladores. Los pipelines de DevSecOps suelen tener un equipo dedicado a la seguridad que define las políticas de la organización. Las cuales, pueden determinar reglas de cifrado, normas de uso de SAST, DST o SCA o mejores prácticas de codificación. Cuando el equipo de desarrolladores tiene claro el conjunto de pautas que deben seguir, conocen en todo momento lo que deben, o no deben hacer, a la vez de como adaptar el trabajo para que la seguridad de las aplicaciones mejore día a día.

Integrar con canalizaciones de CI/CD. El objetivo es descubrir problemas de calidad y generar un ciclo rápido de retroalimentación a los desarrolladores. Este principio se aplica a la seguridad.

- CI/CD debe ejecutar herramientas de seguridad de manera continua y automática.
- Se registran los cambios del código y aplicaciones para obtener métricas e informes.
- Los cambios que generen problemas de seguridad lanzan respuestas automáticas.
- Las herramientas de colaboración utilizadas por el equipo de seguridad deben ser compartidas con los equipos de desarrollo y operaciones.

Resultados en tiempos reducidos. Es conveniente que las herramientas generen resultados prácticamente en tiempo real, ya que la velocidad es otra de las piedras angulares de un equipo DevOps actual.

Alcance extenso y poder de adaptación. Las herramientas de seguridad deben ser capaces de funcionar y no limitar sus capacidades en diferentes entornos, como contenedores cloud, PaaS, Kubernetes, nubes híbridas... Sin generar silos o puntos ciegos.

⁷<https://www.misp-project.org>

A la vez, las herramientas están obligadas a proporcionar información sobre cualquier tipo de aplicación, incluidas las que se basan en código abierto, así como aplicaciones adquiridas de terceros, de las que no se posee su código fuente.

Shift-left y shift-right. Ya se ha comentado en varias ocasiones los beneficios que aporta realizar las evaluaciones de seguridad desde el principio del proyecto, para intentar evitar que posibles vulnerabilidades lleguen a la etapa de producción. Pero también hay que tener en cuenta las últimas etapas de desarrollo (producción) o "desplazamiento a la derecha".

- En producción es donde se generan la mayoría de los ataques.
- El código fuente otorga mucha información. Sin embargo, la que se obtiene mientras la aplicación se ejecuta en el entorno de producción, "vale su peso en oro".
- En algunas ocasiones, aplicaciones ejecutadas en el entorno de producción no se ejecutaron en el de desarrollo, y por tanto, no llegaron a ser analizadas por las herramientas de seguridad específicas.
- Para detectar posibles vulnerabilidades en tiempo real, las aplicaciones deben ser monitorizadas en el entorno de producción.

Precisión. La automatización es clave, pero debe estar acompañada de calidad y precisión. La mayoría de las alertas de seguridad que detectan las herramientas resultan ser falsos positivos, los cuales no requieren ninguna acción. Para que DevSecOps resulte eficaz, hay que recurrir a pruebas que sean capaces de detectar cuándo ocurren los falsos positivos y los falsos negativos, ofreciendo información valiosa al equipo.

Aceptación del desarrollador. Todos los elementos que forman el programa DevSecOps tiene que ser aceptado por los miembros del equipo de desarrollo, ya que tienen que ser capaces de realizar las pruebas necesarias, encontrar vulnerabilidades y poner solución a cualquier problema.

Trazabilidad, auditabilidad y visibilidad. Clave para el éxito de DevSecOps.

- Trazabilidad. Seguimiento de los cambios de configuración y entorno desde la planificación hasta la producción.
- Auditabilidad. Generar de manera automática los informes de los procesos de desarrollo y los controles de seguridad.
- Visibilidad. Comprensión de lo que se ejecuta en el entorno, identificar las amenazas y responder ante ellas.

References

- [1] "Aspectos DevOps | Cátedra Viewnext USAL," Sept. 2023. [Online]. Available: <https://viewnext.usal.es/blog/aspectos-devops>
- [2] "Ciclo de vida de desarrollo de seguridad de Microsoft (SDL) - Microsoft Service Assurance," Sept. 2023. [Online]. Available: <https://learn.microsoft.com/es-es/compliance/assurance/assurance-microsoft-security-development-lifecycle>
- [3] C. Jones, *Applied Software Measurement: Global Analysis of Productivity and Quality*. McGraw Hill, May 2008.
- [4] M. Mark, *Secure, Resilient, and Agile Software Development*. Boca Raton, FL, USA: Auerbach Publications, Sept. 1674. [Online]. Available: <https://www.routledge.com/Secure-Resilient-and-Agile-Software-Development/Merkow/p/book/9781032475011>
- [5] Microsoft, "Controles de DevSecOps - Cloud Adoption Framework." [Online]. Available: <https://learn.microsoft.com/es-es/azure/cloud-adoption-framework/secure/devsecops-controls>
- [6] OWASP, "CLASP - Concepts View," 2006. [Online]. Available: https://owasp.org/www-pdf-archive/Us_owasp-clasp-v12-for-print-lulu.pdf

- [7] M. Souppaya, K. Scarfone, and D. Dodson, “Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities,” *CSRC | NIST*, Feb. 2022.